



Báo cáo Nhập môn Trí tuệ nhân tạo Nhóm 27

Trí Tuệ Nhân Tạo (Trường Đại học Bách khoa Hà Nội)



Scan to open on Studeersnel

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÀI TẬP LỚN

HỌC PHẦN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI: TÌM ĐƯỜNG ĐI CHO MÁY HÚT BỤI

Nhóm : 27
Lớp : 144915
Giảng viên hướng dẫn : TS.Đỗ Tiến Dũng
Danh sách sinh viên thực hiện :

STT	Họ và tên	MSSV	Email
1	Vũ Thành Đông	20215349	dong.vt215349@sis.hust.edu.vn
2	Nguyễn Văn Trường	20215495	truong.nv215495@sis.hust.edu.vn
3	Đỗ Văn An	20215294	an.dv215294@sis.hust.edu.vn
4	Nguyễn Mạnh Hiệp	20215366	hiep.nm215366@sis.hust.edu.vn
5	Nguyễn Lê Đức Anh	20215305	anh.nld215305@sis.hust.edu.vn

HÀ NỘI – THÁNG 12 NĂM 2023

MỤC LỤC

LỜI NÓI ĐẦU.....	2
I. KHẢO SÁT VÀ MÔ TẢ BÀI TOÁN.....	3
1. Giới thiệu và mô tả bài toán.....	4
2. Phương pháp giải quyết bài toán.....	4
2.1. Đối với trường hợp biết vị trí cần hút bụi (bài toán trạng thái đơn). 4	
2.2 . Đối với trường hợp không biết vị trí cần hút bụi (bài toán với sự kiện ngẫu nhiên).....	7
3. Giao diện.....	8
3.1. Giao diện khởi tạo.....	8
3.2. Giao diện người dùng.....	9
3.3 Giao diện khi hút bụi.....	10
4. Ưu điểm của phần mềm.....	11
5. Các vấn đề gặp phải trong giải quyết bài toán.....	12
II. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	13
III. TÀI LIỆU THAM KHẢO.....	14

LỜI NÓI ĐẦU

Trong thế giới đang ngày càng phát triển, sự tiến bộ của công nghệ thông minh đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày. Trong số đó, máy hút bụi thông minh đóng một vai trò quan trọng trong việc giảm bớt gánh nặng công việc nhà cho con người. Để hiểu rõ hơn về cách thức hoạt động của máy hút bụi tự động, bài tập lớn này tập trung vào việc khám phá và phân tích các thuật toán tìm đường đi cho máy hút bụi, từ đó giúp chúng tối ưu hóa công việc làm sạch môi trường sống của chúng ta.

Bài báo cáo này trình bày một ứng dụng dựa trên ngôn ngữ lập trình Python, sử dụng thư viện giao diện người dùng Tkinter cùng với một loạt các thuật toán tìm kiếm như A*, BFS (Breadth-First Search), DFS (Depth-First Search), và các biến thể của chúng. Mục tiêu của ứng dụng này là mô phỏng cách thức máy hút bụi thông minh di chuyển trong một không gian định trước, vượt qua các chướng ngại vật và dọn sạch bụi bẩn.

Qua quá trình phát triển và thử nghiệm, bọn em đã học hỏi được nhiều điều từ việc áp dụng các kiến thức lý thuyết vào thực tế. Bài báo cáo này không chỉ là một bản trình bày kỹ thuật mà còn phản ánh sự sáng tạo, khả năng giải quyết vấn đề, và đam mê công nghệ của nhóm chúng em.

Chúng em hy vọng rằng bài tập này sẽ mang lại cái nhìn sâu sắc về khả năng ứng dụng của thuật toán và lập trình trong việc giải quyết các vấn đề thực tế, đồng thời mở ra những cơ hội mới cho việc phát triển và nâng cao các giải pháp tự động hóa trong tương lai.

I. KHẢO SÁT VÀ MÔ TẢ BÀI TOÁN

1. Giới thiệu và mô tả bài toán.

Bài toán: Tìm đường đi cho máy hút bụi

Bài toán đặt ra để giải quyết vấn đề tìm kiếm đường đi cho máy hút bụi. Điều này đòi hỏi sự kết hợp của các kỹ thuật trong lĩnh vực trí tuệ nhân tạo và thuật toán tìm đường.

- Biểu diễn bài toán
- Dữ liệu đầu vào:
 - Kích thước của căn phòng cần hút bụi (m hàng x n cột)
 - Số vật cản (vị trí vật cản sẽ được đặt ngẫu nhiên)
 - Số địa điểm có bụi cần phải hút (đối với bài toán trạng thái đơn)
- Xử lý dữ liệu
 - Trích xuất dữ liệu: dữ liệu như kích thước của căn phòng (sẽ được chuyển đổi về kích thước của một ma trận $m \times n$ để thuận tiện cho việc triển khai thuật toán), số lượng vật cản được người dùng điền trực tiếp trên giao diện.
 - Thuật toán tìm đường:
 - Thuật toán A*, BFS,DFS cơ bản được sử dụng để tìm đường đi tối ưu cho máy hút bụi với trường hợp máy hút bụi biết vị trí chính xác của địa điểm cần được hút bụi (bài toán trạng thái đơn)
 - Thuật toán BFS,DFS được biến đổi để tìm đường đi cho máy hút bụi với trường hợp máy hút bụi không biết vị trí của địa điểm cần được hút bụi, khi đó máy sẽ tiến hành đi toàn bộ căn phòng, nếu vị trí đó bẩn thì sẽ hút bụi (bài toán có sự kiện ngẫu nhiên)
 - Giao diện người dùng:
 - Chọn điểm xuất phát và đích: người dùng chọn số cột và hàng , số lượng vật cản bằng điền số lượng mong muốn trên trang chính, đồng thời người dùng cũng có thể tùy chọn vị trí của máy hút bụi, vật cản, vị trí cần đường hút bụi bằng cách click chuột vào 1 ô trong bảng.
 - Hiển thị đường đi: Hiển thị đường đi được tìm thấy trên màn hình

2. Phương pháp giải quyết bài toán

2.1. Đối với trường hợp biết vị trí cần hút bụi (bài toán trạng thái đơn)

- Các thuật toán được sử dụng: A*, BFS, DFS
- Thuật toán A (a_star_search):

Mục đích: Tìm đường đi ngắn nhất từ điểm start đến goal trong ma trận.

Phương pháp: Sử dụng hàm heuristic (ước lượng khoảng cách) dựa trên khoảng cách Manhattan. Điểm start được đánh giá với chi phí 0 và được đưa vào hàng đợi ưu tiên.

Quá trình: Thuật toán xét lần lượt từng điểm trong hàng đợi ưu tiên, tính toán chi phí từ điểm hiện tại đến các điểm láng giềng và cập nhật chi phí trên bảng điểm (scores). Nếu điểm hiện tại là điểm đích, thuật toán trả về đường đi.

Xử lý vật cản: Các điểm là vật cản (có giá trị là 1 trong ma trận) sẽ không được xét đến.

❖ Heuristic Function (hàm heuristic):

Mục đích: Tính khoảng cách Manhattan (là tổng giá trị tuyệt đối của hiệu số giữa các tọa độ tương ứng của hai điểm) của a và b.

❖ Tìm Mục Tiêu Gần Nhất (find_closest_goal):

Mục đích: Tìm ra điểm đích goal gần nhất với vị trí hiện tại dựa trên khoảng cách Manhattan.

Quá trình: So sánh khoảng cách từ vị trí hiện tại đến mỗi điểm đích trong danh sách goals.

❖ Tìm Đường Đi Đến Mục Tiêu Gần Nhất (find_path_to_closest_goal):

Mục đích: Tìm đường đi đến điểm đích gần nhất từ vị trí xuất phát start.

Quá trình: Liên tục tìm và di chuyển đến điểm đích gần nhất cho đến khi hết tất cả các điểm đích trong danh sách.

Thuật toán sẽ trả lại một đường đi tối ưu cho máy hút bụi, từ đó máy sẽ đi từng ô trên bản đồ đến khi toàn bộ vị trí có bụi đã được làm sạch.

• Thuật toán BFS (Breadth-First Search):

❖ Hàm check(r, c):

Kiểm tra xem một ô cụ thể trong ma trận có thể đi qua được hay không, từ đó trả về True nếu ô đó không phải là vật cản (biểu diễn bởi '.') và nằm trong giới hạn của ma trận.

❖ Hàm `ste()`:

Xây dựng danh sách các bước đi từ điểm đích đến vị trí bắt đầu dựa trên thông tin lưu trong hàng đợi queue. Danh sách step này chứa trình tự các bước di chuyển.

❖ Hàm `move()`:

- Các bước xử lý của thuật toán BFS.
- Khám phá ma trận từ điểm bắt đầu, mở rộng tìm kiếm đến các ô lân cận cho đến khi tìm thấy điểm đích (dust).
- Khi tìm thấy điểm đích, gọi hàm `ste()` để xây dựng danh sách các bước di chuyển và kết thúc tìm kiếm.

❖ Hàm `initialize_bfs(ta, st, dust)`:

- Khởi tạo và bắt đầu quá trình tìm kiếm.
- Chuyển ma trận đầu vào ta thành một ma trận table với các ô trống và vật cản.
- Bắt đầu tìm kiếm từ vị trí st (start) đến vị trí dust (điểm đích) sử dụng BFS.

❖ Thuật toán sẽ trả về một danh sách step, nó chứa một chuỗi các tọa độ (dưới dạng cặp [hàng, cột]) mô tả đường đi từ điểm xuất phát đến điểm đích. Danh sách này được tạo ra bởi hàm `ste()`, sau khi thuật toán BFS hoàn thành quá trình tìm kiếm thông qua hàm `move()`.

❖ Mỗi phần tử trong danh sách step biểu diễn một bước di chuyển cụ thể trong ma trận. Đường đi này là một dãy các ô liên tiếp trong ma trận, bắt đầu từ vị trí xuất phát và kết thúc tại vị trí đích, đảm bảo rằng mỗi bước chuyển tiếp đều là những bước hợp lệ và không đi qua các ô là vật cản.

• **Thuật toán DFS (Depth-First Search):**

❖ Hàm `check(r, c)`:

- Kiểm tra xem một ô cụ thể tại vị trí (r, c) trong ma trận có thể đi qua được không, sau đó trả về True nếu ô không phải là vật cản và nằm trong giới hạn của ma trận.

❖ Hàm `move(r, c)`:

- Thực hiện việc tìm kiếm chiều sâu từ vị trí (r, c).
- Kiểm tra điều kiện dừng (khi đến được điểm đích dust).
- Thực hiện đệ quy để di chuyển đến các ô lân cận, đánh dấu chúng là đã thăm và cập nhật thông tin đường đi trong ma trận queue.

❖ Hàm initialize_dfs1(ta, st, dust):

- Khởi tạo và bắt đầu quá trình DFS.
- Chuyển đổi ma trận đầu vào ta thành một ma trận table với các ô trống và vật cản, và một ma trận que lưu trữ thông tin đường đi.
- Gọi hàm move để bắt đầu tìm kiếm từ vị trí bắt đầu st (start) đến điểm đích dust.

❖ Thuật toán này trả về một danh sách step, chứa trình tự các bước di chuyển từ điểm xuất phát đến điểm đích. Danh sách này được xây dựng dựa trên thông tin đường đi được lưu trữ trong ma trận que.

❖ Điểm nổi bật của thuật toán này so với thuật toán DFS truyền thống là nó không chỉ thực hiện tìm kiếm theo chiều sâu mà còn lưu trữ thông tin về bước di chuyển để có thể truy ngược lại đường đi từ điểm đích đến điểm bắt đầu. Điều này làm cho thuật toán phù hợp với các tình huống cần xác định rõ ràng lộ trình di chuyển trong một không gian giới hạn.

2.2. Đối với trường hợp không biết vị trí cần hút bụi (bài toán với sự kiện ngẫu nhiên)

- Các thuật toán được sử dụng: BFS, DFS nâng cao
- Thuật toán BFS nâng cao:

❖ Hàm check(r, c, tabl):

- Kiểm tra xem ô tại vị trí (r, c) trong ma trận tabl có thể đi qua được hay không.

=> Trả về True nếu ô không phải là vật cản và nằm trong giới hạn của ma trận.

❖ Hàm ste(que1):

Tạo ra danh sách st chứa các bước di chuyển từ điểm cuối cùng trở lại điểm bắt đầu dựa trên hàng đợi que1, sau đó, đảo ngược danh sách st và thêm nó vào danh sách step để lưu trình tự di chuyển.

❖ Hàm move(sta, dus):

- Thực hiện tìm kiếm BFS từ điểm sta (start) đến dus (dust).
- Sử dụng một hàng đợi que1 để theo dõi các ô đã thăm và một bảng tabl để đánh dấu các ô đã kiểm tra.

❖ Hàm initialize_B1(ta, st):

- Khởi tạo và bắt đầu thuật toán BFS từ điểm bắt đầu st.
- Chuyển đổi ma trận đầu vào ta thành ma trận table với các ô trống và vật cản.

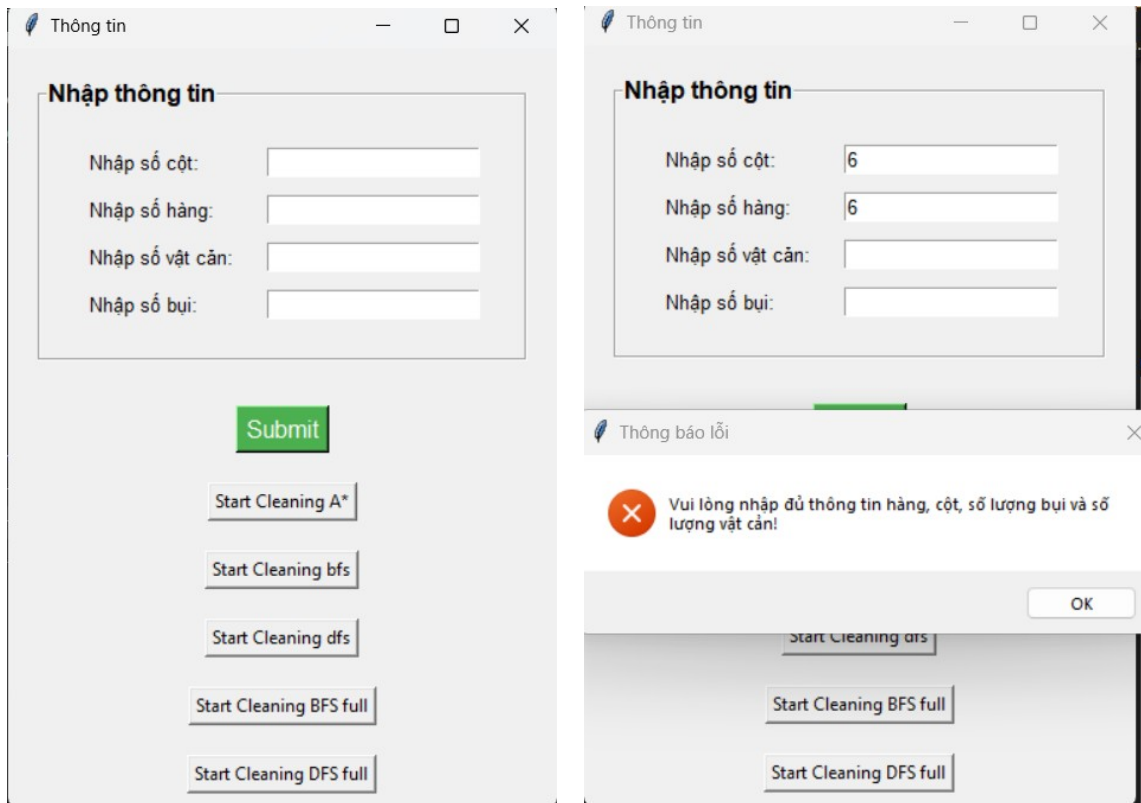
- Gọi hàm move cho mỗi ô trống trong ma trận để tìm đường đến ô đó từ vị trí hiện tại.
- ❖ Kết quả của thuật toán này là danh sách step, chứa trình tự các bước di chuyển từ điểm bắt đầu đến tất cả các ô có thể tiếp cận trong ma trận, không phải là vật cản. Từ đó máy hút bụi sẽ đi hết toàn bộ các ô trong bảng.
- **Thuật toán DFS nâng cao:**
 - ❖ Hàm check(r, c):
 - Kiểm tra xem ô tại vị trí (r, c) có thể đi qua được không, từ đó trả về True nếu ô không phải là vật cản và nằm trong giới hạn của ma trận.
 - ❖ Hàm move(r, c):
 - Hàm đệ quy thực hiện tìm kiếm DFS từ vị trí (r, c).
 - Đánh dấu đường đi và thêm vị trí hiện tại vào danh sách step.
 - Di chuyển đến các ô lân cận theo thứ tự được xác định bởi near và đánh dấu chúng bằng các ký hiệu trong mark.
 - Sau khi trở về từ mỗi lần đệ quy, đánh dấu lại ô đó với ký hiệu đối diện (để cho biết đã quay lại).
 - ❖ Hàm initialize_D(ta, st):
 - Khởi tạo các biến cần thiết và bắt đầu quá trình DFS từ vị trí st.
 - Chuyển đổi ma trận đầu vào ta thành ma trận table với các ô trống và vật cản.
 - Gọi hàm move để bắt đầu quá trình khám phá ma trận từ vị trí bắt đầu.
 - ❖ Thuật toán trả về danh sách step, chứa trình tự các bước di chuyển khám phá toàn bộ ma trận từ vị trí bắt đầu. Mỗi bước trong danh sách này biểu diễn một vị trí trên ma trận mà thuật toán đã đi qua. Điểm đặc biệt của thuật toán này là nó không chỉ thực hiện tìm kiếm theo chiều sâu mà còn đánh dấu lại lộ trình đã đi qua, cho phép ta trực quan hóa quá trình di chuyển trong ma trận.

3. Giao diện

3.1. Giao diện khởi tạo

Người dùng sẽ nhập số cột, số hàng, số vật cản và số bụi theo mong muốn

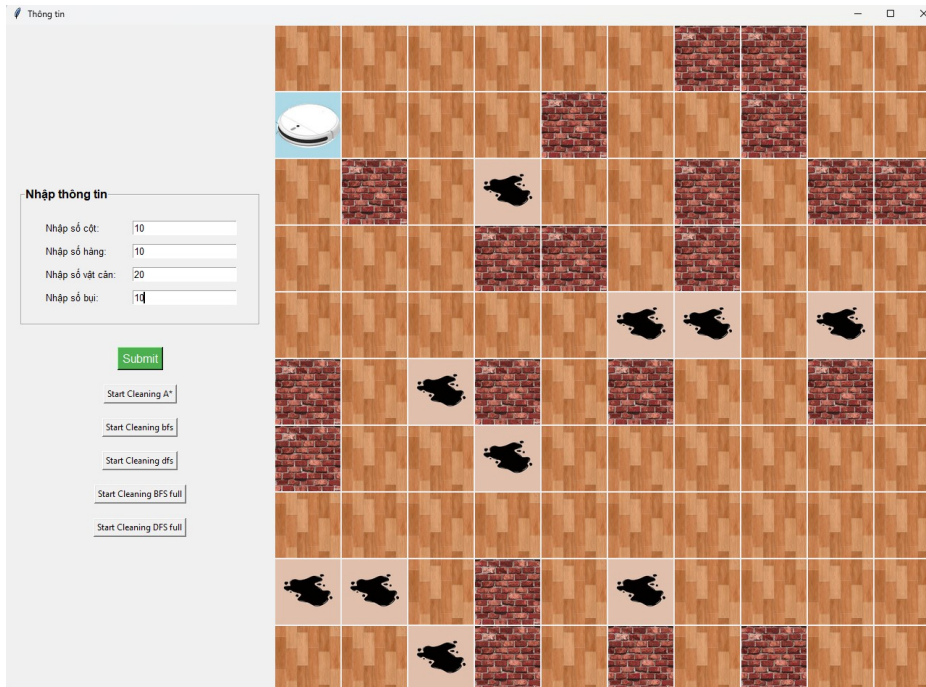
Nếu không nhập đủ, sẽ có 1 messagebox thông báo lỗi



3.2. Giao diện người dùng

Sau khi nhập đủ thông tin, sẽ hiện ra một giao diện gồm có các ô trống biểu thị sàn nhà, các ô biểu thị vị trí cần hút bụi và các ô biểu thị vật cản. Các vị trí này sẽ được hệ thống random ngẫu nhiên.

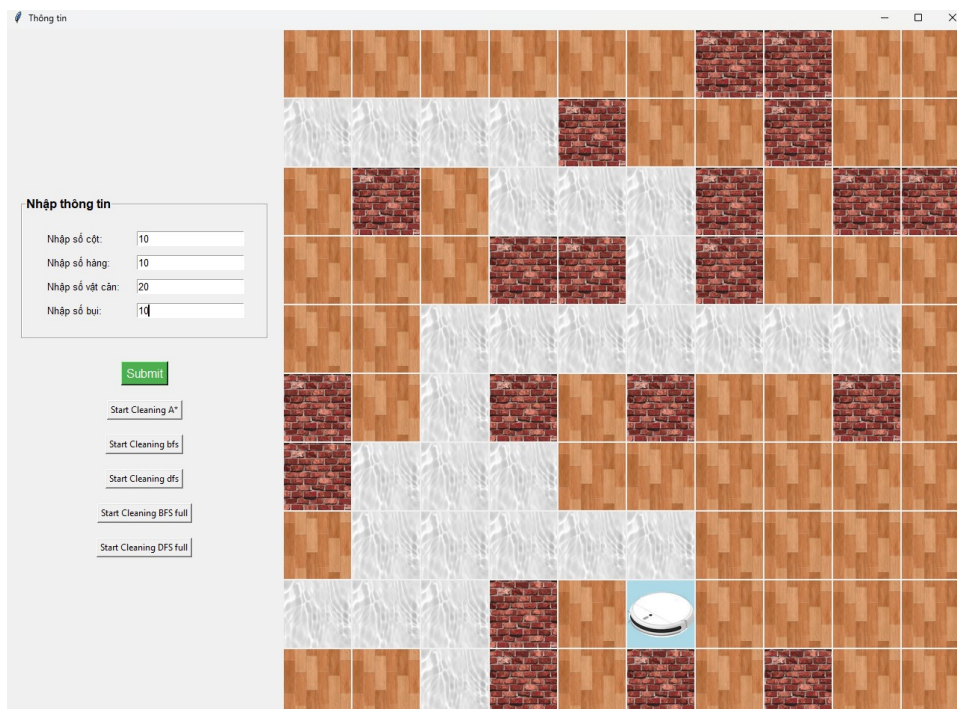
Tuy nhiên, người dùng cũng có thể tùy chọn thêm vị trí của vật cản (bằng cách nhấn nút lăn chuột), thêm vị trí cần hút bụi (nhấn chuột phải) hoặc chọn vị trí bắt đầu của máy hút bụi (nhấn chuột trái)



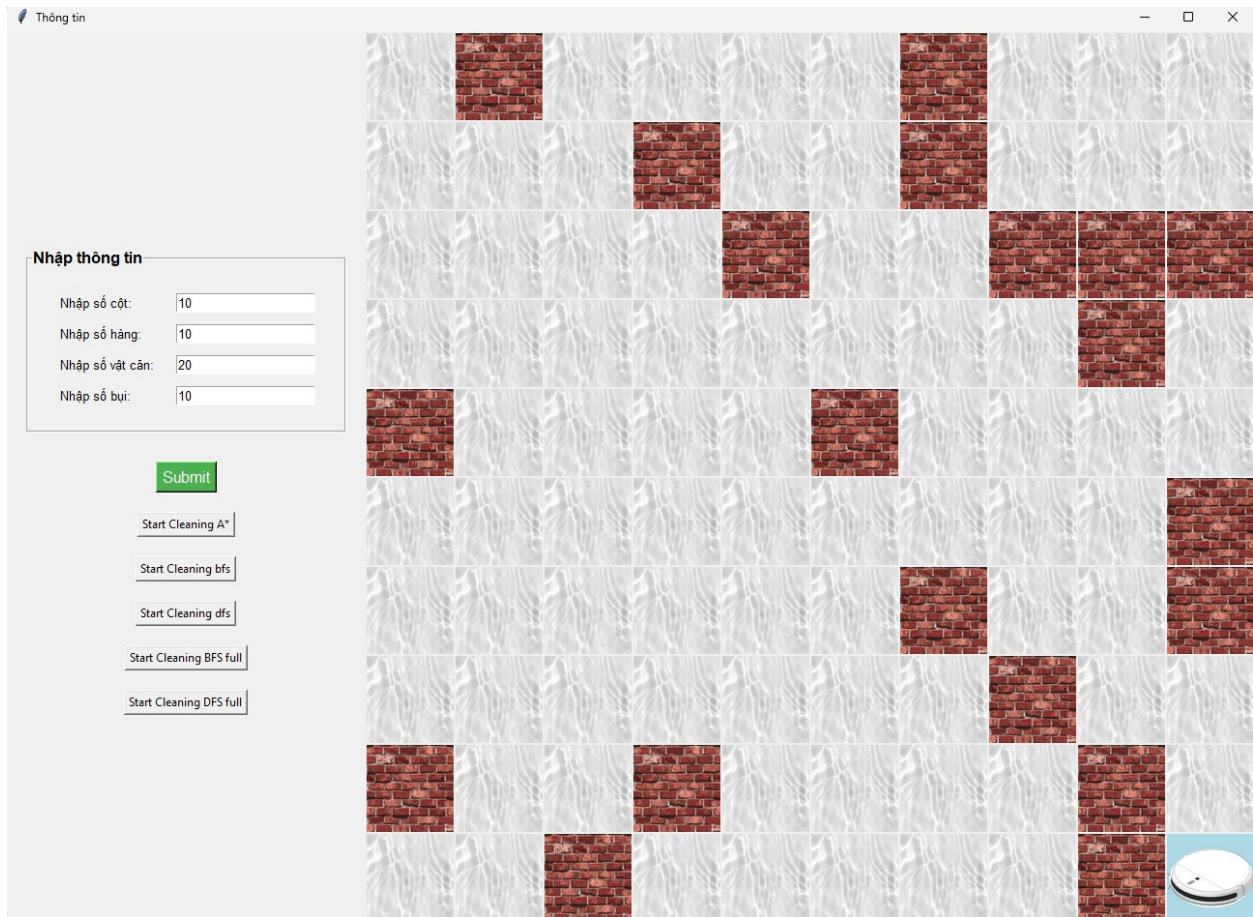
3.3 Giao diện khi hút bụi

Khi chọn bất kì một thuật toán, máy sẽ đi theo đường đi mà thuật toán thực hiện. Ví dụ 2 trường hợp cơ bản:

Đường đi theo thuật toán A*:



Đường đi theo thuật toán BFS nâng cao (máy sẽ đi toàn bộ bảng):



4. Ưu điểm của phần mềm

- **Đa dạng thuật toán tìm đường:** Phần mềm này tích hợp nhiều thuật toán tìm đường như A*, BFS, và DFS, cho phép người dùng lựa chọn phương pháp tối ưu dựa trên nhu cầu và đặc thù của từng tình huống. Điều này giúp tăng tính linh hoạt và hiệu quả trong việc giải quyết các vấn đề tìm đường. Giao diện dễ dàng sử dụng, có thể lựa chọn giữa việc tạo vị trí ngẫu nhiên hoặc tự lựa chọn vị trí máy hút bụi, vật cản, vị trí cần hút bụi theo ý muốn.
- **Giao diện người dùng trực quan:** Sử dụng thư viện Tkinter để tạo giao diện đồ họa, giúp người dùng dễ dàng tương tác và hiểu rõ hơn về quá trình làm việc của phần mềm. Việc trực quan hóa các quá trình tìm đường và các trở ngại cũng giúp tăng tính trực quan và dễ hiểu.
- **Tùy chỉnh linh hoạt:** Người dùng có thể tự tạo ra các bản đồ môi trường với số lượng hàng, cột, vật cản, và điểm bụi tùy ý. Điều này cho phép kiểm tra và đánh giá các thuật toán trong một loạt các tình huống khác nhau.

- **Phản hồi thời gian thực:** Phần mềm cập nhật và hiển thị kết quả ngay lập tức dựa trên các hành động của người dùng, giúp họ dễ dàng nắm bắt được đường đi của máy.
- **Thao tác dễ dàng:** Có thể tùy chọn vị trí mới của máy hút bụi, vật cản và vị trí cần hút bụi thông qua thao tác chuột

5. Các vấn đề gặp phải trong giải quyết bài toán

- Chỉ có thể giải khi có tri thức là bản đồ ngôi nhà (ma trận kích thước $m \times n$).
 - Không tránh được những vật thể có thể di chuyển.
 - Với đầu vào có kích thước hàng và cột lớn sẽ mất nhiều thời gian để tạo giao diện
 - Thuật toán DFS không chạy được với bản đồ lớn và không phải đường đi ngắn nhất
 - Khi chạy 1 thuật toán thì phải tạo lại bảng mới có thể chạy 1 thuật toán mới
- ⇒ Hướng giải quyết: chưa có

II. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong quá trình nghiên cứu và phát triển ứng dụng tìm đường đi cho máy hút bụi, chúng em đã tiến hành mô phỏng và đánh giá hiệu quả của nhiều thuật toán tìm kiếm khác nhau như A*, BFS, DFS và các biến thể của chúng. Kết quả thực nghiệm cho thấy mỗi thuật toán có những ưu và nhược điểm riêng, phụ thuộc vào cấu trúc môi trường làm việc và yêu cầu cụ thể của từng tình huống. A* chứng minh là hiệu quả trong việc tìm kiếm đường đi ngắn nhất, trong khi BFS và DFS cung cấp các phương án thay thế linh hoạt cho các môi trường phức tạp hơn.

Bên cạnh việc hiểu rõ hơn về các thuật toán, dự án này cũng giúp chúng em nhận thức rõ về tầm quan trọng của việc tối ưu hóa và cân nhắc chi phí tính toán trong thiết kế hệ thống tự động. Sự kết hợp giữa lập trình, thuật toán và thiết kế hệ thống đã mở ra một hướng nhìn mới về cách công nghệ có thể được ứng dụng để giải quyết các vấn đề thực tế. Tuy nhiên, do thời gian có hạn nên trong quá trình phát triển cũng còn 1 số phần mà chưa được hợp lý mà chưa thể sửa chữa ngay.

Trong tương lai, nhóm chúng em sẽ cố gắng hoàn thiện phát triển phần mềm để mang lại một phần mềm có trải nghiệm tốt hơn, khắc phục được những nhược điểm bên trên. Nếu có điều kiện cho phép về thời gian, nhân lực nhóm có thể phát triển phần mềm thêm nhiều chức năng khác để giúp đơn giản hóa các công việc được thực hiện thủ công rất mệt mỏi và dễ bị nhầm lẫn.

III. TÀI LIỆU THAM KHẢO

[1] Slide giảng dạy môn “Nhập môn trí tuệ nhân tạo” do thầy Đỗ Tiến Dũng giảng dạy.

Link github phần mềm:

https://github.com/Dong1311/Demo_AI_Vacuum